XS language syntax

1. Variable types

- int
- float
- bool
- string
- vector

```
// Creating a new vector with 0.0/0.0/0.0.
vector v0 = cOriginVector; // Creates a copy of the origin vector.
vector v1 = vector(0.0, 0.0, 0.0);
vector v2 = vector(0.0); // Sets y and z to x.

// Getting vector components.
float a = v.x;
float b = v.y;
float c = v.z;
aiEcho("x: " + v.x);

// Setting vector components works the same way:
v.x = a;
v.y = b;
v.z = c;

// Compound assignments also work as usual:
v.x += a;
v.y -= b;
v.z *= c;

// Vectors can, as shown for v0, be assigned (creating a copy) directly:
vector v3 = myOtherVector; // v3 is now a copy of myOtherVector.

// Vector methods can be called through the . operator:
v = v.normalize(); // Creates a normalized copy of v and overrides the old values.
float l = v.length();
// Note that operations are never in place and return a copy instead that
// you have to reassign.

// Multiplying a vector with a float will multiply all values:
v = v * l; // v.x, v.y, and v.z are multiplied by l and then written back to v.
```

- Built-in methods

    - length
    - normalize
    - dot(vector other)
    - cross(vector other)
    - distance(vector other)
    - distanceXZ(vector other)
    - distanceToLine(vector linePoint, vector direction)
    - distanceToLineSegment(vector p1, vector p2)
    - angleBetweenVector(vector other)
    - angleAroundY()
    - rotateXZ(float angleRadians)
    - translateXZ(float radius, float angleRadians)
- class

```
class Foo
{
   int a = 1;
   int b = 5;
   void(int) c = [](int d = 0) {};

   // member functions have a hidden 'this' parameter, just like other languages.
   void show() { aiEcho("a=" + a + ", b=" + b); }
};

void main()
{
   // foo will have members with values specified in the class declaration.
   Foo foo;
   foo.a = 4;
   foo.show();
}
```

- lambda / Function pointer
  Variable capture is not supported.

```
int add(int a = 0, int b = 0)
{
  return(a + b);
}

void main()
{
   // return type(param type...) varName = [](var declarations...)
   // -> return type { body };
   int(int, int) addOp = [](int a = 0, int b = 0) -> int
   {
      return(a + b);
   };

   // could also be assigned to a regular function.
   addOp = add;

   // default value of 0 is assigned to 'b' parameter.
   int v = add(1);
   // should be a compile error, lambdas don't allow default values.
```

```
   int v = addOp(1);
}

class Addition
{
   // Default we add 10 to the provided number.
   void(int) modifyNumber = [](ref int number) { number += 10) };
}
Addition addition;

void main()
{
   int num = 10;
   addition.modifyNumber(num);
   // num is now 20.

   // Change what modifyNumber does for addition.
   addition.modifyNumber = [](ref int number) -> void
   {
      number += 20;
   };
   int num2 = 10;
   addition.modifyNumber(num2);
   // num2 is now 30.
}
```

- array

```
// default array
int[] arr = default;

// size, default value
arr = new int(10, -1);

// new with class doesn't need a default value parameter,
// as the array elements will all have default values
// provided by the Foo class declaration.
Foo[] fooArr = new Foo(5);
arr[i] = 5;
aiEcho("arr[i]="+arr[i]);
int fooArrSize = fooArr.size();
```

  - Built-in methods + return values.
    - size()
      - num indexes.
    - resize(int size, <type> defaultValue) or just resize(int size) for classes
    - add(<type> value)
      - index at which it was added.
    - uniqueAdd(<type> value) - not supported on classes.
      - index at which it was added.
    - insert(int index, <type> value)
      - index at which it was added.
    - removeIndex(int index)
      - bool on if it was successful or not.
    - removeValue(<type> value) - not supported on classes

        - bool on if it was successful or not.
    - find(<type> value) - not supported on classes
        - index at which it was found, -1 if not found.
    - clear()
- reference

```
// references can only be used on parameters, similar to C#.
// tip: it is recommended to use references with classes or vectors whenever
// possible, because it does avoid doing a copy.
void test(ref int a)
{
  a += 7;
}

void main()
{
  int a = 5;
  test(a);
  // a should be 12.
  aiEcho("a=" + a);
}
```

---

2. Rules

Rules are an XS unique feature that require extensive explanation, as such they have their own document: "*BANG documentation\XS documentation\XS rules functionality*".

---

3. Modifiers

- const - variable cannot be modified once assigned. A const variable must be initialized by another constant variable, XS limitation.
    - const int number = 3; // This is fine.
    - const int number = getNumberFromWherever(); // Compile error.
    - const int number = someOtherVariable; // Compile error.
- static - can only be used inside functions to maintain the value after function exit.
- extern - variable can be used in all source files, global only.
- ref - function parameter modifier, pass by reference instead of value.
- mutable - function modifier, indicates the function can be overwritten with a new definition later with same return and parameter types.

```
// This will compile even though the function is declared after it's being used.
mutable void helperExploreOtherIslands(int planID = -1) {}

void main()
{
   callHelper(15);
}

void callHelper(int planID = -1)
{
   helperExploreOtherIslands(planID);
```

```
}

void helperExploreOtherIslands(int planID = -1)
{
   // Some code.
}
```

---

4. Operators

Not all operators can be used on all combination of variable types. Below is a list of all operators and in what combinations they can be used.

```
int num = 10;
num + 10.0; // This is fine because we add a float to an int, that is supported.
num + vector(0.0, 0.0, 0.0); // This compiler errors because int + vector isn't valid.
```

The variable type on the left indicates the variable type the operator is being called on.
The variable types on the right are the valid types that can be used for this operator + left hand side combination.

Thus if we have an integer and use the + operator on it we can only add ints/floats to it. And if we have a vector and we use the + operator on it we can only add other vectors to it.

- add(+)
  - int + (int/float)
  - float + (int/float)
  - string + (int/float/bool/string/vector)
  - vector + vector
- subtract(-)
  - int - (int/float)
  - float - (int/float)
  - vector - vector
- multiply(*)
  - int * (int/float)
  - float * (int/float)
  - vector *(int/float)
- divide(/)
  - int / (int/float)
  - float / (int/float)
  - vector / (int/float)
- mod(%)
  - int % int
- shift left/right(>>/<<)
  - int >> int
- bitwise and(&)
  - int & int
  - bool & bool
- bitwise or(|)
  - int | int
  - bool | bool

- bitwise xor(^)
  - int ^ int
  - bool ^ bool
- neg(-)
  - -int

All above have compound assignment support, e.g. +=, &=, >>=

- equal(==)
  - int == (int/float)
  - float == (int/float)
  - bool == bool
  - string == string
  - vector == vector
- not equal(!=)
  - int != (int/float)
  - float != (int/float)
  - bool != bool
  - string != string
  - vector != vector
- less(<)
  - int < (int/float)
  - float < (int/float)
  - string < string
- less equal than(<=)
  - int <= (int/float)
  - float <= (int/float)
  - string <= string
- greater(>)
  - int > (int/float)
  - float > (int/float)
  - string > string
- greater equal than(>=)
  - int >= (int/float)
  - float >= (int/float)
  - string >= string
- and(&&)
  - bool && bool
- or(||)
  - bool || bool
- not(!)
  - !(bool)
- ternary operator (condition ? expression1 : expression2)

Tip: Prefer to do operations with the same type

```
float a = 4.0;
// don't write this
```

```
float b = a + 5;
// add the decimal point to indicate a float constant.
float b = a + 5.0;
```

---

5. Conditions

- if

```
if (expression)
{
}
else if (expression)
{
}
else
{
}
```

Tip: prefer fast checks before slow checks so the condition can be short circuited sooner.

```
// when a equals to b, the expression is true and slowCheck() could be ignored
if (a == b || slowCheck() == true)
```

Tip: wrap code behind constant expressions when possible.

```
if (cNumberPlayers == 5)
{
   // do something, this block of code will only be compiled when number of players
   // equals to 5.
   // also equivalent to using #if directives.
}

// this is a partial constant check, someNonConstantCheck() == false will be removed
// when civ equals to Zeus because the expression can be short circuited.
if (cMyCiv == cCivZeus || someNonConstantCheck() == false)

// functions could also be constant checks, such as
bool civNotZeusAndLessThan5Players()
{
   return(cMyCiv != cCivZeus && cNumberPlayers < 5);
}

// And this will be evaluated as compile time.
if (civNotZeusAndLessThan5Players() == false)
```

- switch

```
switch (expression)
{
   case constant:
   {
      break;
   }
   // note: even without break here, we will jump out of constant2 block after
   // execution, which is unlike c++ when the code fall through constant3.
   case constant2:
   {
```

```
   }
   // a case block with multiple values also supported.
   case constant3:
   case constant4:
   {
      break;
   }
}
```

---

6. Loops

- for

```
for (int i = 0; i < 10; i++)
for (i = 10; i >= 0; i -= 2)
//for (initialization; condition; increment)

// tip: don't write this, the function would be called every time during iteration.
for (int i = 0; i < kbBaseGetNumber(cMyID); i++)
// should be like this, saving the function call result into a variable first.
int numBases = kbBaseGetNumber(cMyID);
for (int i = 0; i < numBases; i++)
// however it is fine for non class array arr.size(), due to the call being inlined
// but for class arrays it involves a division which can be a little expensive
// due to the array size internally being stored as array size * class size
for (int i = 0; i < arr.size(); i++)
```

- while

```
while (i < 10)
{
}
```

- do while

```
do
{
} while (i < 10)
```

---

7. Includes

XS supports including files into other files. Each runtime has its own root folder from which all includes are relative to:

- AI: “*game\ai*”
- TR: *“game\data\trigger”*
- RM: “*game\random_maps*”

```
// Even though this also a preprocessor directive we do not use # for includes in XS.
include "core/core.xs";
```

Functions can always be used in other files, as long as their definition is earlier than them being used. You can however use the mutable functionality if you want to use a function before it’s defined.

Variables can not by default be used in other files. You must mark them with the “*extern”* keyword before they can be used in other files. The same principle of definition before usage applies, but you can’t forward declare a variable.

8. Preprocessor directives

```
// Defines a new token.
#define TOKEN_NAME

// Returns true if the specified token has been defined, false otherwise.
// Can also be used at runtime.
defined(token)

// Evaluates a constant expression (can also be a define() directive).
// If the expression evaluates to false, the subsequent codeblock is ignored.
#if (constant expression)
   // Some code...
#elif (constant expression)
   // Some code...
#else
   // Some code...
#endif

// Example using #define:
#define TEST
#if (defined(TEST) == true)
  // Some code that is removed by the preprocessor if the above condition
  // were false (in this example it's true).
#endif
```